

FINAL PROJECT

Campus Exchange Hub

A security-conscious local marketplace prototype

Module: Web Application and Software Architecture · Bachelor · Module B10 · Summer Semester 2026

Scenario: Local Marketplace / Classifieds

Stack: HTML5 · CSS3 · JavaScript · jQuery · JSON · localStorage

This engineering report documents the requirements, architecture, security decisions and implementation of the submitted front-end application.

1. Overview and goals

Campus Exchange Hub is a small classified marketplace for a campus or local community. Students can discover useful second-hand items such as books, furniture, bikes and electronics, while members can publish and manage their own listings.

The Local Marketplace / Classifieds scenario was selected because user-generated content makes validation, safe rendering and role boundaries especially visible. The implementation remains front-end only, as required by the project scope.

Goals

- Provide a clean, responsive browsing experience with search and filtering without page reloads.
- Implement complete Create, Read, Update and Delete operations for Listing records.
- Persist browser-side changes with localStorage.
- Demonstrate Visitor, Member and Moderator interfaces.
- Demonstrate client-side validation and XSS-safe rendering.

The role selector is a demonstration mechanism, not authentication. Trusted authorization and persistence would move to a server in a production system.

2. Requirements gathering

Stakeholders and needs

Stakeholder	Need	Response
Visitor	Find items and contact sellers.	Search/filter, detail, inquiry and report.
Member	Publish and control own listings.	Create/edit/delete, dashboard, favorites.
Moderator	Protect marketplace quality.	Report queue and removal action.
Assessor	See course-topic coverage.	CRUD, jQuery, validation, security, architecture.

User stories

- As a Visitor, I want to search and filter listings so I can find relevant items quickly.
- As a Visitor, I want to open a listing and contact the seller.
- As a Visitor or Member, I want to report a questionable listing.
- As a Member, I want to create, edit and delete my own listings.
- As a Member, I want to save promising listings for later.
- As a Moderator, I want to review reports and remove flagged listings.

Functional requirements

- Dynamic listing read/list/detail views; keyword, category and price filtering.
- Validated listing create/update form with localStorage persistence.
- Member-only ownership controls and dashboard.
- Inquiry form and seller-side dismissal workflow.
- Listing report flow and Moderator queue.
- XSS-safe rendering of user-written content.

3. Architecture and design

The project uses a lightweight separation of concerns. Data handling covers JSON seed loading and localStorage persistence; application logic covers validation, filtering, permissions and CRUD; presentation functions build each page dynamically. Shared logic lives in one JavaScript module while each HTML file provides a clear semantic view.

Data model

Entity	Fields / purpose
Listing	id, title, description, category, price, condition, location, seller, sellerId, createdAt, status, views. Primary CRUD entity.
Inquiry	id, listingId, from, fromId, message, createdAt, status. References a listing.
Report	id, listingId, reason, createdAt, status. References a listing and feeds moderation.
Favorite	Listing IDs stored locally for the current browser/demo user.

Views

View	Purpose
Home	Orientation and entry point.
Browse	Search, filter and scan listing cards.
Detail	Inspect listing, save, inquire or report.
Create / Edit	Member listing CRUD form.
My area	Own listings and incoming inquiries.
Favorites	Saved listings.
Moderation	Moderator report queue.

Design decisions

- Mobile-first layout with CSS Grid/Flexbox and breakpoint rules.
- Navy and warm amber palette with neutral surfaces to keep listing content dominant.
- Role-aware navigation hides tools that are not relevant to the current role.
- Cards use restrained metadata, strong price/title hierarchy and generous spacing.

Real-product storage

In a real deployment, listings, inquiries, reports and user accounts would live in a server-side database behind authenticated API requests. localStorage is intentionally used only as the assignment's browser-side persistence mechanism.

4. Security considerations

4.1 XSS prevention

Listing titles, descriptions, seller names, inquiries and report reasons are created with jQuery elements and inserted with `.text()`. They are therefore treated as text rather than trusted HTML. This avoids turning user-controlled strings into executable markup.

4.2 Input validation

- Title: 4–70 characters; description: 15–600 characters.
- Price: numeric and limited to €0–€100,000.
- Category and condition: allow-listed values.
- Location and seller fields: length and character constraints.
- Inquiry: 10–500 characters.
- Report: required allow-listed reason.
- Price filters reject an inverted minimum/maximum range.

4.3 Role boundaries

Visitors cannot use the Member create/edit tools through the normal interface. Members can edit/delete only listings whose `sellerId` matches the demo Member. Moderators receive the report queue and can remove reported listings. These are browser-side demonstrations, not secure authentication.

4.4 Production limitation

Client-side checks can be bypassed. A real product must repeat validation, sanitization and authorization on the server, use real authentication, HTTPS, secure session/token handling, suitable CSRF protections, database constraints and abuse/rate controls.

5. Implementation

Technologies and file organization

File	Purpose
index.html	Landing page.
browse.html	Searchable/filterable listings.
detail.html	Listing detail, inquiry and report flow.
create.html	Create/edit listing form.
dashboard.html	Member workspace and inquiries.
favorites.html	Saved listings.
moderator.html	Moderator report queue.
css/style.css	External responsive styling.
js/app.js	Data loading, validation, permissions, CRUD and UI rendering.
data/seed.json	Initial JSON dataset.

Requirement mapping

Requirement	Implementation evidence
Semantic HTML / forms	header, nav, main, section, footer, labels, skip link and visible focus states.
External responsive CSS	Single stylesheet using Grid/Flexbox and media queries.
JavaScript validation / rendering	Central validate() plus dynamic render functions.
Search/filter	Input/change handlers update results immediately.
jQuery	Events, DOM creation, slideToggle report form, fade toast and \$.getJSON().
CRUD + persistence	Listing create/read/update/delete stored in localStorage.
Roles	Visitor, Member and Moderator selector with role-aware UI and permissions.
Security	User text rendered with .text(); constrained input and production limitations documented.
Minimum views/forms	Seven views and listing, filter, inquiry and report forms.

Notable implementation decisions

- The JSON seed is loaded with \$.getJSON() when served over HTTP; a small fallback keeps file:// demonstrations usable.
- localStorage data is centralized through one persistence layer so reload behaviour is predictable.
- Dynamic content is built from DOM elements instead of trusting user-provided HTML.
- The report flow uses a collapsible form and feeds the Moderator queue.
- The interface stays deliberately lightweight rather than introducing a framework that is not required by the module.

6. Conclusion and future work

The final prototype covers the required front-end topics: semantic HTML, responsive external CSS, JavaScript validation and rendering, jQuery interaction, JSON loading, localStorage persistence, complete Listing CRUD, role-specific interfaces, multiple forms, inquiry/report workflows and XSS-aware rendering.

The main simulated elements are authentication, trusted authorization and server-side persistence. Those limitations are intentional and should be replaced before any real deployment.

Future work

- Real account authentication and server-side authorization.
- Backend API and database for listings, inquiries, reports and favorites.
- Safe image upload and storage.
- Pagination, rate limiting, audit logging and stronger moderation.
- Automated tests and accessibility testing with assistive technologies.
- Git-based version control and deployment automation.

Demonstration checklist

- Visitor: search/filter listings and open a detail page.
- Visitor/Member: send an inquiry and submit a report.
- Member: create a listing, reload, edit it, then delete it.
- Member: save and remove a favorite; review incoming inquiries.
- Moderator: open the report queue and remove the flagged listing.
- Code review: show validation, role checks, localStorage and .text()-based rendering.

Submission contents

The final package contains the runnable website, organized source files, seed JSON, README and this technical report. No server-side language or database is required for the assignment.